ComNetX: Local Hierarchical Adaptation for Dynamic Community Detection

Anonymous Anonymous Anonymous Anonymous Anonymous
Anonymous Anonymous Anonymous Anonymous Anonymous

***Abstract*—Dynamic community detection is commonly addressed either by full-snapshot recomputation or by solver-specific dynamic procedures. Full recomputation preserves the semantics of mature static solvers, but it repeatedly processes unchanged graph regions when updates are small. Solver-specific dynamic methods can reduce this cost, but their update rules often have limited transferability across objectives, feature representations, and implementations. In addition, localizing computation only by graph distance may omit community context needed by high-quality solvers. We introduce ComNetX, a solver-agnostic hierarchical adaptation framework for local dynamic updates. ComNetX maintains a multi-level community state, expands the updated region, closes it over affected communities, and contracts these communities into compact local instances. This affected-community closure and contraction preserve solver context while restricting computation to the changed part of the graph. The same interface can wrap modularity heuristics, graph-clustering models that use node features, and native dynamic solvers as local backends. We evaluate ComNetX through a multi-backend study on six real networks, longer real-data streams for topology-based backends, and controlled dynamic stochastic block model (DSBM) stress streams. The results show that ComNetX can preserve the quality of strong modularity-based solvers while reducing update time on large graphs: in paired runs on the largest real graph, Local Leiden keeps final modularity within 0.006 of full-snapshot recomputation while achieving a $41.9 \pm 0.2\times$ speedup. The combined protocols also identify regimes where locality breaks down and a full refresh is preferable.**

***Index Terms*—dynamic graphs, community detection, graph clustering, modularity, local algorithms, graph neural networks**

I. Introduction

Dynamic networks model systems whose interactions change over time, including social, communication, biological, and information networks [1]–[3]. Community detection (CD) is a central task in such data: it identifies groups of vertices with coherent structural or functional roles and supports monitoring, recommendation, anomaly detection, and exploratory analysis [4]–[7]. Modularity remains one of the most widely used quality criteria [8], [9], and scalable methods such as Louvain and Leiden are strong practical baselines [10], [11]; shared-memory implementations further improve static throughput [12], [13]. However, these methods were mainly designed for static snapshots. In dynamic settings, even a small batch of edge updates may require rerunning a solver on the entire graph, which is prohibitive for large networks.

Existing dynamic CD methods reduce this cost by reusing previous partitions, applying incremental modularity updates, or processing affected neighborhoods [3], [14]–[18]. These ideas are essential for scalability, but they are often tied to a specific objective or update rule. Meanwhile, modern graph clustering increasingly incorporates attributes and neural representations [19]–[24]. Feature-aware and GNN-based [25] methods can capture richer dependencies than purely topological heuristics, yet repeatedly applying them to full snapshots is even more expensive. A dynamic framework should therefore support both classical and feature-aware solvers without redesigning each solver from scratch. Continuous-time temporal community methods address a related but different problem: they track communities directly in link streams and model when vertices enter or leave them. We discuss this line in Section V, but the target of ComNetX is batched snapshot maintenance for large dynamic graphs.

The key challenge is locality. If a batch update modifies only a small part of the graph, the algorithm should inspect and update only a bounded neighborhood of the affected vertices. This is natural for real-time graph analytics, but preserving the quality of a strong base solver under local processing is nontrivial: an overly small subgraph can lose important community context, whereas an overly large one eliminates the computational benefit.

In this paper, we present *ComNetX*, a local hierarchical framework for dynamic community detection. ComNetX maintains a hierarchy of communities, identifies the communities affected by a batch update, constructs compact aggregated subproblems, and invokes the selected solver only on these local instances. The aggregation step preserves surrounding community context while avoiding full-snapshot recomputation. When a solver uses node attributes, the same aggregation pattern is applied to the feature matrix, making the framework compatible with topology-only and feature-aware methods. In this way, ComNetX acts as an algorithmic layer around existing CD solvers rather than as a solver tied to a single objective.

Our contributions are:

1) We formalize a unified dynamic adaptation setting for arbitrary community detection solvers and contrast naive full-snapshot recomputation with local processing.
2) We introduce ComNetX, a hierarchical local wrapper that aggregates affected communities, transfers features when required, and projects updated labels back to the original graph.

3) We analyze the update cost in terms of the affected neighborhood, the touched communities, and the contracted backend instances.
4) We evaluate the framework with a layered protocol: a broad multi-backend compatibility study, longer real-data streams, and controlled dynamic stochastic block model (DSBM) stress streams that separate random, hub-centered, and community-internal updates.

The paper is organized as follows. Section II gives the preliminary definitions. Section III presents the ComNetX methodology and analysis. Section IV reports the experiments. Section V reviews related work, and Section VI concludes the paper.

II. Preliminaries

A. Dynamic Graph Model

Let $G_t = (V, E_t, \mathbf{A}_t, \mathbf{X})$ denote a graph snapshot at discrete time t. We assume a fixed vertex universe V with $n = |V|$ vertices throughout the stream. Edge updates may make a vertex isolated in a particular snapshot, or make a previously isolated vertex incident to new edges, but such vertices remain elements of V. The weighted adjacency matrix is $\mathbf{A}_t \in \mathbb{R}^{n\times n}$. When node attributes are available, they are stored in $\mathbf{X} \in \mathbb{R}^{n\times d}$; otherwise the method can run with synthetic or no features, depending on the selected backend.

The transition from G_{t-1} to G_t is represented by a signed batch update Δ_t:

$$\mathbf{A}_t = \mathbf{A}_{t-1} + \Delta_t.$$

A positive entry inserts or increases an edge weight, while a negative entry deletes or decreases it. The endpoints of nonzero entries form the directly affected set

$$S_t = \{\, i \in V \mid \exists j : \Delta_t(i,j) \neq 0 \text{ or } \Delta_t(j,i) \neq 0 \,\}.$$

For directed input graphs, ComNetX uses the symmetrized adjacency only for detecting affected neighborhoods; the stored adjacency and the downstream metric computation keep the representation selected by the experiment. In the implementation, $\mathbf{A}_t$ and Δ_t are stored in sparse matrix formats.

B. Community Detection Interface

A community detection solver is treated as a black-box procedure

$$\mathcal{B}(\mathbf{A}, \mathbf{X}, \mathbf{y}_0) \mapsto \mathbf{y},$$

where $\mathbf{A}$ is an adjacency matrix, $\mathbf{X}$ is optional, and $\mathbf{y}_0$ is an optional warm-start partition. This interface covers modularity heuristics such as Leiden [11] and FLMIG [26], feature-aware GNN clustering models such as DMoN [24], MAGI [27], and S^{2}CAG [28], and dynamic backends such as MFC [29] and DF-Leiden [18] when they are used as local solvers. A *naive* dynamic adaptation of $\mathcal{B}$ simply runs it on each full snapshot:

$$\mathbf{y}_t = \mathcal{B}(\mathbf{A}_t, \mathbf{X}, \mathbf{y}_{t-1}).$$

This is often a strong quality baseline, but its update time depends on the full graph rather than on the changed region.

C. Modularity

For topology-based evaluation we use modularity [8]. For a weighted adjacency matrix $\mathbf{A}$, directed or undirected, let $W = \sum_{i,j} A_{ij}$, $d_i^{\text{out}} = \sum_j A_{ij}$, and $d_j^{\text{in}} = \sum_i A_{ij}$. For partition labels c_i, we compute

$$Q = \frac{1}{W} \sum_{i,j} \left[A_{ij} - \gamma \frac{d_i^{\text{out}} d_j^{\text{in}}}{W} \right] \mathbf{1}\{c_i = c_j\},$$

where γ is the resolution parameter. For an undirected graph stored as a symmetric adjacency, $W = 2m$ and $d_i^{\text{out}} = d_i^{\text{in}} = d_i$, which recovers the standard undirected definition. We use $\gamma = 1$ in the main experiments and report a resolution-sensitivity check in the ablation study. Modularity is not a ground-truth measure and is affected by known resolution limits [30], [31]; therefore we also report normalized mutual information (NMI) as a compact external-agreement measure whenever labels are available. For a predicted partition C and ground-truth labels Z, we use the symmetric normalization

$$\text{NMI}(C, Z) = \frac{2I(C;Z)}{H(C) + H(Z)},$$

where I is empirical mutual information and H is empirical entropy of the corresponding label distributions [32].

III. Methodology

A. Overview

ComNetX is a local adaptation layer around a base solver $\mathcal{B}$. It does not change the objective optimized by $\mathcal{B}$. Instead, after each batch update it constructs a compact local instance that preserves the community context around affected vertices. The maintained state consists of

- the accumulated sparse adjacency matrix $\mathbf{A}_t$;
- the feature matrix $\mathbf{X}$ when the backend requires features;
- a hierarchy $\mathbf{C}_t \in \mathbb{Z}^{L\times n}$, where $\mathbf{C}_t[\ell, i]$ is the community label of vertex i at hierarchy level ℓ.

Figure 1 summarizes the update workflow. A new batch is first added to the accumulated graph, its endpoints are expanded to a small graph neighborhood, and then ComNetX reoptimizes only the communities intersecting that region. At each level, these communities are contracted into supernodes, the base solver is executed on the contracted graph, and the result is mapped back to the original vertices.

B. Affected Region and Hierarchical Closure

Let r be a small integer radius. ComNetX expands the directly affected set S_t to

$$B_r(S_t) = \{v \in V \mid \text{dist}_{\mathbf{A}_t}(v, S_t) \leq r\}.$$

Here $\text{dist}_{\mathbf{A}_t}(v, S_t) = \min_{u\in S_t} \text{dist}_{\mathbf{A}_t + \mathbf{A}_t^\top}(v, u)$, i.e., the shortest-path distance to the set S_t in the graph induced by nonzero entries of the symmetrized adjacency.

The radius-expanded affected set alone is not sufficient for high-quality local recomputation: if only part of an existing community is included, the local solver loses the context needed to decide whether that community should split, merge,

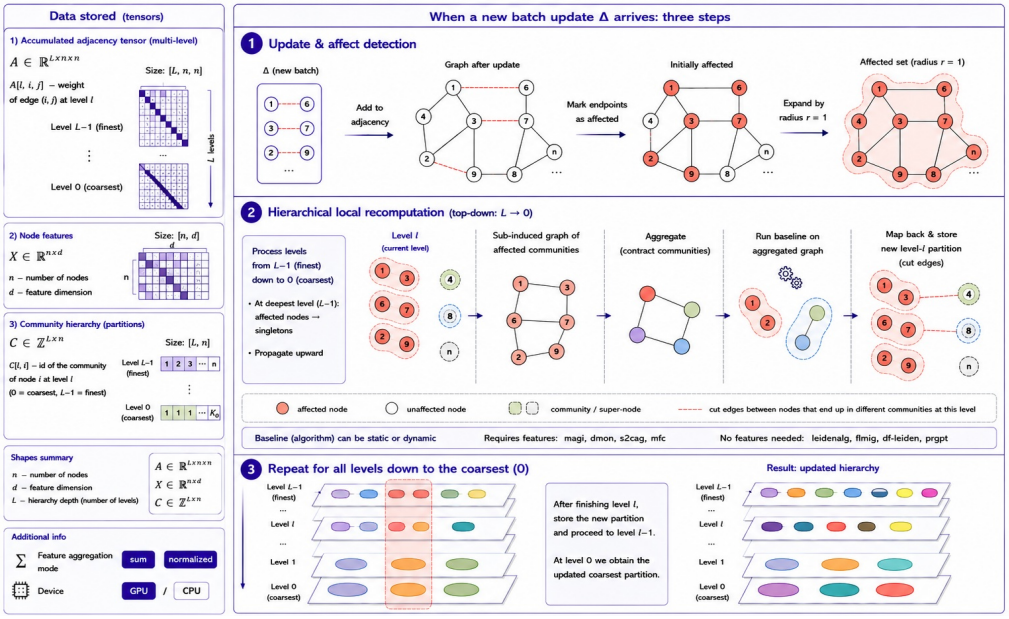

Fig. 1. ComNetX update workflow: affected vertices are expanded, affected communities are contracted into local subproblems, the selected backend is run locally, and updated labels are written back to the hierarchy.

or remain unchanged. Therefore, at each hierarchy level ℓ we close the affected set under the current community labels:

$$U_t^\ell = \{v \in V \mid \exists u \in B_r(S_t) : \mathbf{C}_{t-1}[\ell, v] = \mathbf{C}_{t-1}[\ell, u]\}.$$

Only vertices in U_t^ℓ may be relabeled at level ℓ; vertices outside this set keep their previous labels. Thus radius expansion identifies the communities that may need revision, and hierarchical closure supplies the full current context of those communities to the local backend.

C. Local Aggregation

Fix the update time t. At level ℓ, let $g_\ell(i) \in \{1, \dots, k_\ell\}$ be the compact identifier of the current level ℓ community containing $i \in U_t^\ell$. ComNetX builds a sparse membership matrix $\mathbf{P}_\ell \in \{0,1\}^{k_\ell \times n}$:

$$\mathbf{P}_\ell[a, i] = 1 \quad \Longleftrightarrow \quad i \in U_t^\ell \text{ and } g_\ell(i) = a.$$

Let $\mathbf{A}_t^{(0)}$ be $\mathbf{A}_t$ restricted to the affected level 0 closure and embedded in the original $n \times n$ index space. During the transition from level ℓ to level $\ell + 1$, ComNetX removes the edges of $\mathbf{A}_t^{(\ell)}$ whose endpoints belong to different level ℓ communities; the resulting working adjacency is $\mathbf{A}_t^{(\ell+1)}$. The local contracted graph at level ℓ is then

$$\bar{\mathbf{A}}_\ell = \mathbf{P}_\ell \mathbf{A}_t^{(\ell)} \mathbf{P}_\ell^\top,$$

so edge weights between two supernodes are exactly the sums of edge weights between the corresponding level ℓ communities.

If the selected backend uses features, ComNetX uses normalized aggregation for feature-aware local runs. Let $n_{\ell a} = |\{i \in U_t^\ell : g_\ell(i) = a\}|$; then

$$\bar{\mathbf{X}}_\ell[a] = n_{\ell a}^{-1} \sum_{i \in U_t^\ell : g_\ell(i) = a} \mathbf{X}_i.$$

For topology-only solvers, feature aggregation is skipped. Structural edge weights are preserved in $\bar{\mathbf{A}}_\ell$; in the reported feature-aware runs they do not additionally reweight $\bar{\mathbf{X}}_\ell$.

D. Relabeling and Hierarchy Update

The contracted instance also receives a warm-start partition from the previous hierarchy. Let $\bar{\mathbf{y}}_\ell^0[a]$ be the previous level ℓ label of the vertices represented by supernode a, renumbered to compact labels. Backends that do not support warm starts ignore this argument. The base solver is invoked on the aggregated instance:

$$\bar{\mathbf{y}}_\ell = \mathcal{B}(\bar{\mathbf{A}}_\ell, \bar{\mathbf{X}}_\ell, \bar{\mathbf{y}}_\ell^0).$$

The aggregated labels are projected back to the original graph by assigning all vertices represented by the same supernode to the corresponding new local cluster. ComNetX reuses existing community identifiers whenever possible, which avoids unnecessary relabeling of unchanged regions.

After updating level ℓ, ComNetX removes from the working adjacency all edges whose endpoints now lie in different communities at that level. This "cut" operation ensures that the next hierarchy level refines only the interior of the communities produced so far. Before the level loop begins, vertices in the expanded affected set are opened as singletons at the deepest stored level, and affected higher level communities inherit the finer labels. This gives the backend enough freedom to split a changed region while keeping the unaffected part of the hierarchy fixed. Algorithm 1 summarizes the update procedure for one batch.

E. Complexity and Locality

Let $b_t = |B_r(S_t)|$, let $h_\ell = |U_t^\ell|$, and let k_ℓ be the number of affected communities contracted at level ℓ. Let $\mathrm{nnz}(\mathbf{M})$ denote the number of nonzero entries in a sparse matrix $\mathbf{M}$.

Algorithm 1 ComNetX update for one batch

Require: accumulated graph $\mathbf{A}_{t-1}$, hierarchy $\mathbf{C}_{t-1}$, features $\mathbf{X}$, update Δ_t, radius r, backend $\mathcal{B}$

$\mathbf{A}_t \leftarrow \mathbf{A}_{t-1} + \Delta_t$
$S_t \leftarrow$ endpoints of nonzero entries in Δ_t
$B \leftarrow B_r(S_t)$
open vertices in B as singletons at the deepest hierarchy level
propagate finer labels inside affected higher level communities
for each hierarchy level ℓ **do**
    $U_t^\ell \leftarrow$ vertices in communities intersecting B
    build sparse membership matrix $\mathbf{P}_\ell$
    $\bar{\mathbf{A}}_\ell \leftarrow \mathbf{P}_\ell \mathbf{A}_t^{(\ell)} \mathbf{P}_\ell^\top$
    aggregate $\bar{\mathbf{X}}_\ell$ if $\mathcal{B}$ requires features
    derive warm start $\bar{\mathbf{y}}_\ell^0$ from previous level ℓ labels
    $\bar{\mathbf{y}}_\ell \leftarrow \mathcal{B}(\bar{\mathbf{A}}_\ell, \bar{\mathbf{X}}_\ell, \bar{\mathbf{y}}_\ell^0)$
    map $\bar{\mathbf{y}}_\ell$ back to vertices in U_t^ℓ
    cut inter-community edges in $\mathbf{A}_t^{(\ell)}$
end for
return updated hierarchy $\mathbf{C}_t$

For one update, ComNetX performs neighborhood expansion, sparse aggregation, and one backend call per level. Its time can be written as

$$O\Big(\operatorname{nnz}(\Delta_t) + \operatorname{expand}_r(S_t) + \sum_{\ell=0}^{L-1} \big[\operatorname{nnz}(\mathbf{A}_t[U_t^\ell]) + T_{\mathcal{B}}(k_\ell, \bar{m}_\ell, \bar{d}) \big] \Big).$$

where $\mathbf{A}_t[U_t^\ell]$ denotes the principal submatrix of $\mathbf{A}_t$ induced by U_t^ℓ, $T_{\mathcal{B}}(k_\ell, \bar{m}_\ell, \bar{d})$ is the running time of the base solver on the contracted instance, $\bar{m}_\ell = \operatorname{nnz}(\bar{\mathbf{A}}_\ell)$, and $\bar{d}$ is the aggregated feature dimension when features are used. The memory footprint of a backend call is governed by the contracted local graph, not by the full snapshot.

For $r = 1$, the expanded set satisfies $b_t \le |S_t| + \sum_{v \in S_t} d_t(v)$, where d_t is the degree in the symmetrized expansion graph; hence hub endpoints can destroy locality even when the number of changed edges is small. More generally, b_t is controlled by the degree profile around S_t and is upper-bounded only by $|V|$ without assumptions on degree distribution. If $\mathcal{P}_\ell$ is the previous level ℓ partition, then $h_\ell = \sum_{C \in \mathcal{P}_\ell : C \cap B_r(S_t) \neq \varnothing} |C|$ and $k_\ell \le \min(b_t, |\mathcal{P}_\ell|)$, so closure is governed by the sizes of the communities touched by the expanded set. Finally, $\bar{m}_\ell \le \operatorname{nnz}(\mathbf{A}_t[U_t^\ell])$ because contraction can only merge edges between the same pair of supernodes. Thus the useful bounds are instance-dependent: in the worst case a high-degree update or a giant affected community can make U_t^ℓ comparable to V, and ComNetX degenerates toward full recomputation. The observable fractions $b_t/|V|$, $\max_\ell h_\ell/|V|$, and $\max_\ell \bar{m}_\ell / \operatorname{nnz}(\mathbf{A}_t)$ therefore give a practical guardrail: if the expanded, closed, or contracted local instance exceeds a predefined computational budget, the update should be handled by full recomputation. The experiments below use $r = 1$ and $L = 3$, a configuration selected because larger radii quickly increase the inspected region on the studied graphs while depth values below three reduce the benefit of the hierarchy.

IV. Experiments

This section evaluates ComNetX as a local adapter for static and dynamic community detection solvers. We compare it with full-snapshot recomputation and with native dynamic baselines, focusing on the quality–time trade-off after a sequence of graph updates. The experiments are organized around five questions:

- **RQ1:** Does ComNetX preserve community quality while reducing update time relative to full-snapshot recomputation?
- **RQ2:** Can the same local adaptation improve or complement native dynamic community detection algorithms?
- **RQ3:** How much of the graph is inspected after neighborhood expansion, community closure, and contraction?
- **RQ4:** Which components–radius expansion, hierarchy depth, feature representation, closure, and contraction–control the quality–efficiency trade-off?
- **RQ5:** How stable is the method under different update streams, longer horizons, and adversarially large affected regions?

A. Experimental setup

1) Datasets and Evaluation Protocols: We evaluate on six real-world networks summarised in Table I. The selected datasets provide ground-truth labels, which allows us to complement modularity with external clustering metrics. For this study all graphs are converted to an undirected representation by symmetrizing the adjacency matrix. This keeps the input format uniform across baselines, including methods whose original implementations do not support directed updates.

The evaluation is split into the following three protocols.

P1: multi-backend compatibility study. For each real network, the edge sequence is partitioned into 1000 coarse batches. The first 999 batches form the initial graph G_{999}, and a reference partition $P(G_{999})$ is computed with Leiden [11]. The last coarse batch is then processed as ten chronological mini-batches. This 999:10 protocol is used for broad backend coverage, including expensive GNN baselines.

P2: real-data batch-sensitivity and long-horizon runs. A strategy p:q partitions the chronological edge stream into $p+1$ coarse batches, builds the initial graph from the first p batches, computes the initial partition on that graph, and splits the last coarse batch into q mini-batches. We use 999:50 and 999:100 to refine the same tail as P1 into more update steps, and 9:500 to test a longer horizon from a much earlier initial graph.

P3: controlled DSBM stress streams. The synthetic protocol uses a dynamic stochastic block model (DSBM) construction based on the stochastic block model (SBM), a standard generative model for graphs with block-dependent edge probabilities [33]. In the dynamic setting, the graph is observed as a sequence of snapshots generated under a time-indexed block structure [34]. We instantiate this idea with an initial planted-partition graph followed by controlled

update layers. The reported 100-batch rows vary the update rate and whether changed edges are random, hub-centered, or community-internal; different streams can therefore end at graphs with different affected-region structure.

TABLE I
REAL-WORLD DATASETS.

Dataset	Nodes	Edges	Q_{GT}
dyn_cora	2 708	5 278	0.64
dyn_acm	3 025	13 128	0.48
dyn_citeseer	3 327	4 552	0.54
patent	12 214	41 916	0.35
dyn_pubmed	19 717	44 338	0.43
arxivmath	270 013	799 745	0.64

The ground-truth partitions have lower modularity than the strongest modularity-optimizing outputs on these graphs. Thus, moderate external agreement should be interpreted together with the structural objective: high modularity partitions may refine or reorganize label classes rather than matching them exactly.

All wall-clock measurements were collected on a Dockerized Linux server with two AMD EPYC 7742 64-core CPUs, 2.0 TiB RAM, and eight NVIDIA A100-SXM4 GPUs (80 GB each). The NVIDIA driver was 535.216.03, `nvidia-smi` reported CUDA 12.2, and the container used Python 3.10.13, PyTorch 2.3.1 with CUDA 11.8, and TensorFlow 2.14.0.

2) Metrics: We evaluate both community quality and computational efficiency.

- **Modularity** (Q): the standard modularity of the partition obtained after processing the update sequence, measured on the final graph of the corresponding protocol.
- **External agreement**: normalized mutual information (NMI) is computed against the ground-truth labels on the final graph of each protocol. We report NMI as a compact external metric because it is label-permutation invariant and remains comparable when the number of detected communities differs from the number of label classes.
- **Structural quality**: we compute final-partition mean conductance and normalized cut for full and Local Leiden on the two largest `999:10` streams. These metrics are complementary diagnostics, not optimization targets.
- **Total processing time** (T): the cumulative wall-clock time (in seconds) required to process all measured update mini-batches, i.e., the sum of the execution times of the dynamic algorithm over the whole update sequence. Data-format conversion time is excluded; each baseline operates on its native graph representation.

Whenever repeated measurements are available, comparisons are paired by dataset, method, and batch strategy. Table II gives the P1 compatibility summary, and Table III reports five paired repetitions on the two largest P1 streams.

3) Baselines: We compare against static full-snapshot solvers, static graph-clustering models adapted through the same snapshot interface, and native dynamic baselines. All methods are evaluated on the same symmetrized graph sequence for each dataset. *Leidenalg* [11] is the main topology-only static baseline and represents strong modularity optimization under full recomputation. *FLMIG* [26] is included as a Louvain-style local move method with aggressive pruning. For representation-learning and feature-aware graph clustering, we include DMoN [24], MAGI [27], S^{2}CAG [28], and the PRGPT-Infomap and PRGPT-Locale variants [35]; DMoN, MAGI, and S^{2}CAG use node features when available, whereas the PRGPT variants are evaluated through their topological clustering outputs. The native dynamic baselines are DF-Leiden [18], an OpenMP-parallel dynamic Leiden implementation, and MFC [29], a feature-aware GNN-based dynamic community detection method. Thus, Leidenalg, FLMIG, PRGPT-Infomap, PRGPT-Locale, and DF-Leiden are treated as topology-only baselines in our evaluation, while DMoN, MAGI, S^{2}CAG, and MFC are feature-aware baselines. To keep the measurements reproducible, every empirical baseline must be callable from the same Python-controlled batched workflow, consume the same edge-update sequence, and return a complete hard partition for each measured snapshot. Several related dynamic methods discussed in Sec. V are therefore treated as Related Work rather than empirical baselines because their available implementations require external non-Python execution paths, target local/overlapping/continuous-time communities, or do not expose the same full-partition output protocol. In particular, we also attempted to run the available Java implementation of DynaMo [16] on the selected graph streams, but it did not complete the common protocol reproducibly.

For algorithms that involve iterative training, the configured budgets are 100 epochs for MFC, one iteration for FLMIG, and 10 epochs for S^{2}CAG, MAGI, and DMoN. These values match the experimental configuration used for the reported runs. MAGI and DMoN are not given ground-truth cluster counts. For MAGI, after learning node embeddings, we select the number of clusters with a vMF-mixture elbow criterion and then apply k-means; when a previous partition is available, its number of labels is used only as a warm-start estimate. For DMoN, whose output dimension is fixed in advance, we set this dimension to the number of nodes in the current graph and compact the nonempty argmax clusters after inference.

Focused comparison design. The compatibility study covers all six real datasets and all implemented baselines with comparable outputs. Focused tables use complete two-dataset blocks on *dyn_pubmed* and *arxivmath*. Leidenalg, DF-Leiden, and S^{2}CAG respectively represent full recomputation, a native dynamic topology solver, and feature-aware graph clustering under runtime pressure. Other backends are retained in the breadth study to document compatibility and sensitivity across a wider method family. Synthetic DSBM streams are used only for operating-envelope stress tests, and continuous-time temporal-community methods are treated as complementary in Related Work.

4) Configuration of the proposed local adaptation: Our dynamic adapter uses three hierarchy levels and a graph-neighborhood radius of one, meaning that the endpoints of the update are expanded to their immediate graph neighbors before community closure is applied. Separate neighborhood-size measurements show that larger radii can quickly approach

TABLE II
COMPATIBILITY STUDY ACROSS BACKENDS.

Backend	Base	ΔQ	ΔNMI	Geometric mean	arxivmath
Leidenalg	Naive	-0.013	-0.012	8.0×	41.9×
FLMIG	Naive	-0.247	-0.039	60.5×	2606.8×
PRGPT-Infomap	Naive	-0.131	-0.064	7.7×	45.2×
PRGPT-Locale	Naive	-0.272	-0.083	8.9×	62.1×
S^2CAG	Naive	$+0.137$	$+0.117$	5.0×	10.2×
MAGI	Naive	$+0.081$	-0.035	0.3×	OOM→985.0s
DMoN	Naive	$+0.514$	$+0.113$	1.0×	OOM→90.0s
MFC	Dynamic	$+0.075$	$+0.136$	56.8×	OOM→381.7s
DF-Leiden	Dynamic	-0.010	$+0.001$	0.1×	8.4×

TABLE III
REPEATED 999:10 ROBUSTNESS.

Dataset	Q_L / Q_B	T_L / T_B
Leiden, Local versus full recomputation		
dyn_pubmed	0.776 (0.000) / 0.786 (0.000)	0.39 (0.00) / 6.68 (0.02)
arxivmath	0.881 (0.000) / 0.886 (0.000)	3.56 (0.02) / 149.07 (0.72)
DF-Leiden, Local versus native dynamic backend		
dyn_pubmed	0.779 (0.000) / 0.762 (0.000)	0.24 (0.00) / 0.11 (0.00)
arxivmath	0.880 (0.000) / 0.850 (0.000)	0.45 (0.01) / 3.77 (0.04)
S^2CAG, dataset features		
dyn_pubmed	0.530 (0.000) / 0.622 (0.000)	26.44 (3.32) / 111.19 (1.28)
arxivmath	0.089 (0.000) / 0.001 (0.000)	225.51 (6.82) / 2297.03 (16.33)

the full graph on several datasets; this motivates the radius-one default.

Feature-aware local configurations use normalized aggregation. The reported S^2CAG compatibility row uses dataset features, while Table V separately evaluates dataset, random, and one-hot representations.

B. Experimental results

1) Compatibility Study Across Backends: This experiment addresses RQ1. Table II summarizes runtime coverage across all protocol P1 datasets. Geometric mean speedups exclude zero-time and out-of-memory entries. For each method, we compare the conventional baseline ("Naive" full recomputation or a native "Dynamic" solver) with the variant wrapped by the local hierarchical adapter. In the full-snapshot setting, all arxivmath P1 rows complete except MFC, MAGI, and DMoN; the Local adapter makes those three rows finite as well. The long-horizon and stress tests below then assess stability as streams lengthen or locality weakens.

Modularity and external agreement. The ΔQ and ΔNMI columns of Table II report aggregate P1 quality deltas, computed as the adapter result minus the baseline result over finite baseline runs. These values show that Local Leiden closely tracks full-snapshot recomputation, while FLMIG and the PRGPT variants trade larger speedups for lower aggregate quality. The feature-aware rows are more heterogeneous: S^2CAG and DMoN improve both aggregate quantities under P1, MAGI mainly contributes out-of-memory avoidance on the largest graph with mixed NMI behavior, and MFC improves the aggregate P1 quality measures while still remaining below the best topology-based methods in absolute quality. The focused repeated rows in Table III and the S^2CAG feature-mode ablation in Table V provide the visible quality breakdowns for the largest datasets and feature choices.

Reduction in processing time. The runtime advantage of the Local adapter is substantial for the larger full-recomputation baselines. For Leidenalg, Table II reports a $41.9\times$ speedup on *arxivmath*; Table III reports the paired repeated estimate, together with $17.1 \pm 0.2\times$ on *dyn_pubmed*. FLMIG and the PRGPT variants obtain even larger or consistent large-graph speedups, but their quality deltas show that this speed comes from a more aggressive loss of global context. The GNN rows show a different benefit: S^2CAG becomes substantially faster, while DMoN and MAGI become executable on large graphs because only small neighborhood graphs are materialized on the GPU; DMoN is near break-even in aggregate runtime but avoids the full-snapshot out-of-memory failure on *arxivmath*. The DF-Leiden row shows that locality can also help an algorithm already designed for dynamic updates when the graph is large enough.

2) Statistical Robustness: This experiment addresses the repeated-run part of RQ5. Table III reports measurements where Local and baseline methods share the same dataset, method settings, and batch strategy. Each row fixes the algorithm parameters; the standard deviations therefore reflect five paired repetitions of the same `999:10` stream shape. The Local rows use the fixed three-level, radius-one configuration and are compared with the corresponding naive or native dynamic baseline. Table entries use the compact mean(sd) form for Local/Baseline quantities.

This repeated-run view shows that topology-only conclusions are stable under a fixed stream shape: Leiden keeps a small modularity gap and large speedups, while Local DF-Leiden is useful on *arxivmath* but not on *dyn_pubmed*, where the native dynamic backend is already efficient. The S^2CAG dataset-feature rows show that the adapter can also accelerate a feature-aware backend with real attributes: on *arxivmath*, runtime drops by $10.2\times$ and modularity improves over full-snapshot S^2CAG; on *dyn_pubmed*, the speedup is smaller and the quality gap larger.

Direction-preserving Leiden check. The cross-backend protocol symmetrizes real graphs because several baselines are undirected. To check that this does not hide a Leiden-specific failure mode, we rerun Leiden on the original directed *dyn_pubmed* and *arxivmath* streams. The directed check shows the same qualitative pattern as the symmetrized rows: on *dyn_pubmed*, full versus Local Leiden gives $Q = 0.788$ versus 0.768, NMI 0.208 versus 0.199, and 4.99s versus 0.36s ($14.0\times$); on *arxivmath*, the corresponding values are $Q = 0.886$ versus 0.880, NMI 0.415 versus 0.404, and 108.82s versus 2.83s ($38.4\times$).

Structural-quality metrics. For the symmetrized `999:10` protocol used in the main real-data comparison, we additionally compute conductance and normalized cut for full versus Local Leiden on the two largest streams. These metrics do not cover every backend, but they test the main topology comparison. Local Leiden reduces normalized cut and leaves mean conductance essentially unchanged or slightly lower: *dyn_pubmed* changes from mean conductance/Ncut $0.128/6.17$ to $0.127/5.33$, and *arxivmath* from $0.0011/8.73$

to $0.0009/6.96$.

3) Comparison with Native Dynamic Methods: This experiment addresses RQ2 by comparing adapted static algorithms with native dynamic solvers DF-Leiden and MFC on the same batched streams and final-snapshot metrics. The comparison is an operating-envelope result, not a universal dominance claim. In the paired `999:10` rows, Local DF-Leiden improves modularity on both large datasets and has nearly neutral aggregate NMI, whereas the 500-update horizon in Table VII is more conservative: it remains faster but loses final NMI on both datasets.

DF-Leiden is already highly optimized, so the Local wrapper is not always the fastest choice. It is slower on *dyn_pubmed*, but reaches $8.4\pm0.1\times$ speedup on *arxivmath* and remains $7.4\times$ faster at the 500-update horizon in Table VII. MFC is slower and runs out of memory on the largest dataset in the full dynamic setting. Thus, native dynamic solvers remain important speed baselines, and ComNetX is most useful when a small affected region would otherwise trigger a much larger backend call.

4) Locality and Workload Reduction: This experiment addresses RQ3. The main mechanism behind the speedup is that ComNetX transforms a full-snapshot backend call into a sequence of small local calls. Table IV reports the measured growth of update neighborhoods in the real-data streams. Since ComNetX uses $r = 1$ in the main configuration, the most important column is $|B_1(S_t)|/|V|$; the larger radii show how quickly locality would be lost if the expansion radius were increased. Here $|S_t|$ is the average number of directly affected vertices. The B_h columns report mean vertex percentages within distance h from S_t.

The measured neighborhoods support $r = 1$: the average radius-one region is below 5% on all datasets and below 1% on *dyn_pubmed* and *arxivmath*. Larger radii quickly erode locality: $r = 3$ already inspects about 27–31% of vertices on these two graphs, and $r = 4$ exceeds half of them. The default therefore uses immediate neighborhoods plus hierarchical community closure rather than deeper graph-radius expansion.

The second locality stage is hierarchical closure and contraction, summarized by U_t^ℓ and $(k_\ell, \bar{m}_\ell)$. For the P1 `999:10` workload profiles, Figure 2 plots the final-level contracted edge fraction against Base/Local time ratio, with marker area showing the mean final-level closure fraction $|U_t^{L-1}|/|V|$ and C, P, A denoting *dyn_cora*, *dyn_pubmed*, and *arxivmath*. Backends receive sub-percent or near-percent edge fractions, but speedups remain backend-dependent: DF-Leiden already performs local dynamic updates, whereas S^{2}CAG still executes neural feature materialization and tensor operations inside each call.

TABLE IV
EMPIRICAL NEIGHBORHOOD GROWTH.

Dataset	$\lvert S_t\rvert$	B_1	B_2	B_3	B_4
dyn_cora	2.0	0.35	1.75	7.77	24.72
dyn_acm	5.2	2.04	8.11	21.61	40.60
dyn_citeseer	2.0	1.01	3.06	5.82	10.13
patent	5.1	4.49	5.46	18.57	18.61
dyn_pubmed	8.8	0.79	7.28	30.58	62.82
arxivmath	145.6	0.95	7.01	27.36	56.60

The workload profiles separate locality from overhead: expansion, closure, aggregation, and projection remain in the millisecond range, while backend execution dominates Leiden on *arxivmath* and all S^{2}CAG rows. For S^{2}CAG, the dominant cost is the neural backend itself: feature materialization, sparse tensor construction, and training. On A100 GPUs with 80 GB memory, full-snapshot neural runs often fit in memory, so GPU parallelism can partly mask graph-size reduction.

5) Ablation Study: This experiment addresses RQ4. The ablation suite isolates the tunable choices evaluated in this study: graph-radius expansion, hierarchy depth, feature representation, affected-community closure, and contraction. Radius and depth are measured with Leiden to isolate the topology-only part of ComNetX, while feature representation choices are measured with S^{2}CAG. The closure and contraction mechanism is first quantified through Table IV and Figure 2, and then tested directly in Table VI. For the topology ablation, Figure 3 plots the complete $L \times r$ grid measured for Leiden on three representative `999:10` streams. Each point gives final modularity Q versus speedup over full-snapshot Leiden; higher and farther right is better.

Figure 3 shows that the topology parameters are not monotone quality knobs. On *dyn_pubmed*, endpoint-only settings dominate the high-radius rows: $L = 2, r = 0$ gives the best local modularity ($Q = 0.783$, $94.5\times$), while $L = 1, r = 1$ remains fast ($54.4\times$) with less quality loss than the default. On *arxivmath*, the $r \geq 1$ rows have nearly the same quality ($Q \approx 0.881$), but the runtime cost grows sharply with radius; $L = 1, r = 0$ is much faster ($634.3\times$) with a slightly lower $Q = 0.879$. An additional *dyn_cora* grid gives yet another pattern, with $L = 3, r = 0$ closest in modularity and $L = 2, r = 0$ the fastest near-quality point. Together with the different neighborhood-growth rates in Table IV, these measurements indicate that the useful L, r range is topology-dependent and that no reliable monotone selection rule was observed in this study. The default $L = 3, r = 1$ is therefore a fixed cross-experiment configuration rather than an estimated optimum. For unseen streams, a conservative budgeted protocol is to test small-radius, shallow configurations on a pilot segment, increase radius or depth only when quality improves while the workload fractions remain well below the full graph,

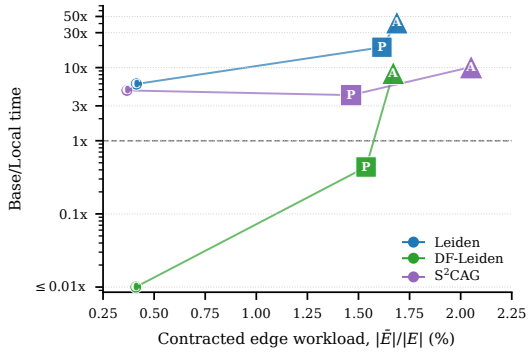

Fig. 2. P1 contracted workload and speedup.

and otherwise fall back to full recomputation.

We also test whether the local contraction changes the behavior of Leiden's resolution parameter. On the `999:50` *dyn_pubmed* and *arxivmath* streams, we rerun full and Local Leiden for $\gamma \in \{0.5, 1, 2\}$. Local speedups remain large across the sweep: 41.6–44.8× on *dyn_pubmed* and 135.9–161.8× on *arxivmath*. The modularity gaps $Q_{\text{Local}} - Q_{\text{Full}}$ are -0.004, -0.026, and -0.053 on *dyn_pubmed*, and -0.004, -0.010, and -0.013 on *arxivmath* for $\gamma = 0.5$, 1, and 2, respectively. Thus the runtime advantage is not specific to $\gamma = 1$, while higher resolution can increase quality sensitivity on some graphs. We therefore keep a fixed resolution for paired comparisons and leave adaptive resolution selection for local instances outside the present scope.

Table V reports the S^{2}CAG feature-mode ablation under the P2 `999:100` strategy. Each cell is final modularity Q, cumulative time T, and final-snapshot NMI. The compact random feature Local row is close to the dataset-feature Local row while being cheaper, which makes it a reasonable default when the original attributes are unavailable or costly.

TABLE V
S^{2}CAG 999:100 FEATURE MODES.

Variant	dyn_cora Q / T / NMI	dyn_pubmed Q / T / NMI
Naive, dataset features	0.691 / 35.71 / 0.437	0.624 / 372.45 / 0.202
Naive, random features	−0.001 / 13.02 / 0.023	−0.002 / 92.18 / 0.000
Naive, one-hot features	0.751 / 43.96 / 0.360	0.528 / 8568.07 / 0.059
Local, dataset features	0.788 / 6.82 / 0.463	0.532 / 36.54 / 0.180
Local, random features	0.784 / 4.67 / 0.462	0.534 / 25.68 / 0.186
Local, one-hot features	0.779 / 7.25 / 0.452	0.518 / 195.59 / 0.182

Table V gives two cautions. First, real features can help: on *dyn_cora*, the local dataset-feature row has the best local modularity and NMI, and Table III shows accelerated dataset-feature S^{2}CAG on *arxivmath*. Random features are therefore not a replacement for attributes; they nearly collapse full-graph S^{2}CAG and become competitive only locally, where topology dominates smaller contracted instances. Second, one-hot features improve naive modularity on *dyn_cora*, but are too expensive on *dyn_pubmed* and do not offset their cost locally. In Table VI, "No closure" keeps only the radius-expanded vertices, whereas "No contraction" gives the backend the post-closure induced subgraph without community contraction; T is the total profiled time in seconds.

The direct ablation separates the two roles of the hierarchy. On *dyn_cora*, removing closure or contraction can reduce time when overhead dominates, but no contraction creates an instance about 12× larger than the contracted one and loses modularity. On larger graphs, removing closure is fastest but loses substantial modularity; removing contraction keeps quality close on *arxivmath*, but grows the backend instance by two orders of magnitude. Contraction is therefore what turns closure from context recovery into a scalable update.

TABLE VI
CLOSURE AND CONTRACTION ABLATION FOR LEIDEN.

Variant	Dataset	Q	T	$\lvert\bar{V}\rvert/\lvert V\rvert$
Full ComNetX	dyn_cora	0.811	0.54	0.42%
Full ComNetX	dyn_pubmed	0.745	0.67	0.20%
Full ComNetX	arxivmath	0.877	5.75	0.24%
No closure	dyn_cora	0.794	0.11	0.35%
No closure	dyn_pubmed	0.565	0.76	0.18%
No closure	arxivmath	0.731	5.17	0.22%
No contraction	dyn_cora	0.776	0.27	4.92%
No contraction	dyn_pubmed	0.667	10.23	8.71%
No contraction	arxivmath	0.874	956.40	33.02%

6) Real-Data Long-Horizon Robustness: This experiment addresses the drift component of RQ5. Protocol P2 evaluates dynamic stability by varying the initial-history fraction and the number of update batches. Table VII reports paired 9:500 topology measurements for *dyn_pubmed* and *arxivmath*. Rows give final modularity, final-snapshot NMI, cumulative time, and Local speedup over the corresponding non-local backend.

Local Leiden is about 16× faster than full Leiden on *dyn_pubmed* and about 31× faster on *arxivmath*. The NMI gap also grows relative to the ten-update setting, especially on *dyn_pubmed*; drift therefore must be reported explicitly rather than inferred from short streams.

TABLE VII
LONG-HORIZON TOPOLOGY ENDPOINTS.

Dataset	Method	Updates	Q	NMI	T	Speedup
dyn_pubmed	Leiden	500	0.784	0.208	315.38	1.0×
dyn_pubmed	Local Leiden	500	0.748	0.153	19.78	15.9×
dyn_pubmed	DF-Leiden	500	0.762	0.195	6.23	1.0×
dyn_pubmed	Local DF	500	0.715	0.138	5.94	1.05×
arxivmath	Leiden	500	0.887	0.427	7201.70	1.0×
arxivmath	Local Leiden	500	0.874	0.355	235.84	30.5×
arxivmath	DF-Leiden	500	0.855	0.388	198.86	1.0×
arxivmath	Local DF	500	0.869	0.335	26.84	7.4×

7) Controlled DSBM Stress Test: This experiment addresses the failure-mode component of RQ5 by varying update size and touched-vertex type in an independent DSBM stress suite. Each synthetic sequence starts from an SBM-style planted-partition snapshot and then applies controlled edge updates to produce dynamic graph snapshots. Each graph has $n = 100{,}000$ vertices, $k = 32$ planted communities, target average degree 58, 1% degree-512 hubs, and assortativity $p_{\text{in}} = 0.97$. We evaluate random, hub-centered, and community-internal updates over 100 batches at changed-edge rates of 0.01% and 0.05%, with five seeds for each locality/rate pair.

The DSBM stress study exposes the operating envelope. Table VIII summarizes the mean Local speedup and the worst quality loss across five seeds per setting. At the smaller update rate, all three locality patterns are faster than full recomputation. At 0.05%, random and hub-centered streams fall below break-even, while community-internal streams remain faster. Larger or hub-heavy batches should therefore trigger a full refresh.

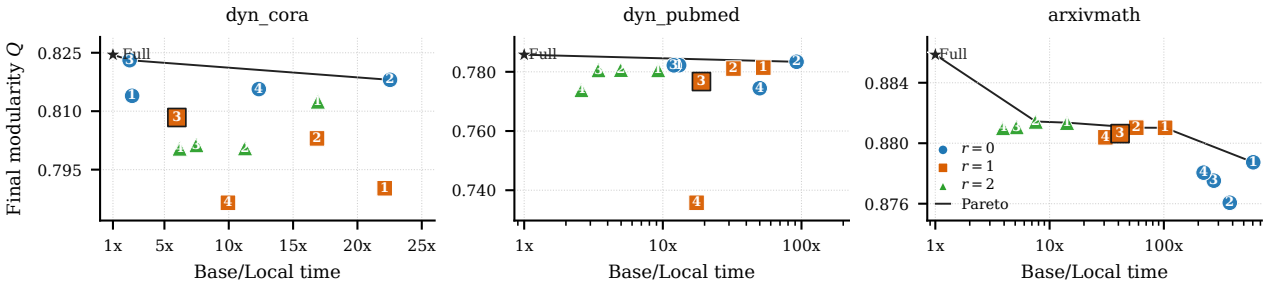

Fig. 3. Quality–time frontier for the Leiden topology ablation. Marker style encodes the graph radius r, the marker number is the hierarchy depth L, and the black line marks nondominated points.

TABLE VIII
CONTROLLED DSBM STRESS SUMMARY.

Rate	Type	Speedup	Worst ΔQ	Worst ΔNMI
0.01%	random	3.34×	−0.0031	−0.0158
0.01%	hub	1.53×	−0.0022	−0.0135
0.01%	community	4.29×	−0.0102	−0.0481
0.05%	random	0.89×	−0.0049	−0.0329
0.05%	hub	0.85×	−0.0050	−0.0286
0.05%	community	1.13×	−0.0088	−0.0561

8) Limitations: ComNetX targets streams where affected communities remain small relative to the graph. Hub-heavy or very large updates may approach the full snapshot or lose boundary context, and representation-learning backends are more sensitive than Leiden to restricted context and feature representation; monitored workload and stress-test diagnostics therefore indicate when a full refresh is preferable. The cross-backend rows use explicit symmetrization for methods that require undirected graphs; the direction-preserving Leiden experiment supports the same trend, but directed baselines beyond Leiden and adaptive local resolution remain future work.

V. RELATED WORK

Static and modularity-based community detection. Classical CD methods rely on modularity, information flow, label propagation, or related structural criteria. Louvain and Leiden are widely used because they combine high modularity with strong scalability [10], [11], while Infomap and label-propagation methods offer alternative flow- and propagation-based views [5], [36], [37]. The quality and limitations of such partitions have been extensively studied, including resolution effects, ground-truth evaluation, and robustness of detected communities [4], [31], [38], [39]. Recent static methods continue to improve modularity optimization through greedy disassembly and local-move strategies [26], [40], but direct application to evolving graphs still tends to require full recomputation.

Dynamic and local community detection. Dynamic CD methods balance snapshot quality, temporal consistency, and update efficiency [3], [15], [41], [42]. Incremental modularity and label-propagation algorithms update selected parts of the previous solution [14], [16], [43]–[47], while flow- and evolution-based approaches track persistent structures over time [48]–[50]. Continuous-time objectives, including longitudinal modularity and LAGO, avoid temporal discretization [51]; our setting instead targets batched snapshot maintenance and reuse of arbitrary solvers under local updates. Other work studies sparse temporal observations, fully dynamic graph routines, and scalable multicore or distributed variants [52]–[57]. Locality appears in several different regimes. Chong and Teow propose batch incremental maintenance [14]; adaptive label propagation, Delta-Screening, and dynamic Leiden variants update affected regions through solver-specific rules [17], [18], [47]. DiTursi et al., Christopoulos et al., and Liu et al. study local, anchor-centered, or seed-based community discovery in dynamic networks [58]–[60], where the output is tied to query or seed communities rather than a full partition of every snapshot. OLCPM addresses online overlapping communities [61]. These methods are important predecessors for local dynamic processing, but their objectives and output protocols differ from the solver-agnostic full-partition maintenance studied here. ComNetX instead wraps arbitrary snapshot and dynamic solvers through contracted local subproblems and evaluates the same adapter across topology-only and feature-aware backends.

Attributed and GNN-based graph clustering. Attributed and GNN-based methods extend CD beyond purely topological criteria. Surveys and recent models show that neural embeddings can capture high-order dependencies and combine naturally with modularity or clustering losses [19], [22], [24], [62], [63]. Examples include modularity-oriented contrastive learning and neural modularity maximization [27], [64], structural-entropy objectives [65], pre-train-and-refine pipelines for scalable partitioning [35], [66], and spectral subspace clustering for attributed graphs [28]. Recent models also explore fairness-aware deep community detection [67]. Although these methods improve expressiveness, they are often expensive to rerun after each graph update.

Overall, prior work offers either efficient routines specialized to a fixed dynamic design or expressive static and feature-aware solvers that are costly to rerun. ComNetX is complementary: it wraps static, GNN-based, and native dynamic solvers while restricting computation to affected aggregated subproblems.

VI. CONCLUSION

We studied the trade-off between reusable full-snapshot community detection and efficient solver-specific dynamic updates. Full recomputation preserves solver semantics but

revisits unchanged graph regions; specialized dynamic algorithms reduce update cost but are less transferable across objectives and implementations; and purely neighborhood-based localization can lose the community context required by strong solvers. ComNetX addresses these limitations through a solver-agnostic hierarchical adaptation mechanism: it preserves a community hierarchy, closes each affected region over the communities that provide context, contracts that region into a compact backend instance, and projects the updated labels back to the full graph. The main contribution is a reusable local-update layer that allows topology-only, feature-aware, and native dynamic solvers to operate on contracted affected subproblems through a common interface. The experiments show that this design preserves much of the modularity of full recomputation for Local Leiden while reducing update time on larger networks; ablations show that radius, hierarchy depth, and feature representation materially affect the trade-off; and long-horizon and DSBM stress tests show when cumulative speedups persist or a full refresh is preferable.

REFERENCES

[1] J. Onnela, J. Saramäki, J. Hyvönen, et al., "Structure and tie strengths in mobile communication networks," *Proc. Natl. Acad. Sci. USA*, vol. 104, no. 18, pp. 7332–7336, 2007.

[2] G. Palla, A.-L. Barabási, and T. Vicsek, "Quantifying social group evolution," *Nature*, vol. 446, pp. 664–667, 2007.

[3] G. Rossetti and R. Cazabet, "Community discovery in dynamic networks: A survey," *ACM Comput. Surveys*, vol. 51, no. 2, Art. no. 35, 2018.

[4] M. A. Porter, J.-P. Onnela, and P. J. Mucha, "Communities in networks," *Notices Amer. Math. Soc.*, vol. 56, no. 9, pp. 1082–1097, 2009.

[5] S. Fortunato, "Community detection in graphs," *Phys. Rep.*, vol. 486, nos. 3–5, pp. 75–174, 2010.

[6] F. Radicchi, C. Castellano, F. Cecconi, V. Loreto, and D. Parisi, "Defining and identifying communities in networks," *Proc. Natl. Acad. Sci. USA*, vol. 101, no. 9, pp. 2658–2663, 2004.

[7] G. W. Flake, S. Lawrence, C. L. Giles, and F. M. Coetzee, "Self-organization and identification of web communities," *Computer*, vol. 35, no. 3, pp. 66–71, 2002.

[8] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Phys. Rev. E*, vol. 69, Art. no. 026113, 2004.

[9] U. Brandes, D. Delling, M. Gaertler, R. Görke, M. Hoefer, Z. Nikoloski, and D. Wagner, "On modularity clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 20, no. 2, pp. 172–188, 2008.

[10] V. Blondel, J. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech.: Theory Exp.*, vol. 2008, no. 10, Art. no. P10008, 2008.

[11] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," *Sci. Rep.*, vol. 9, no. 1, Art. no. 5233, 2019.

[12] H. Lu, M. Halappanavar, and A. Kalyanaraman, "Parallel heuristics for scalable community detection," *Parallel Comput.*, vol. 47, pp. 19–37, 2015.

[13] S. Sahu, K. Kothapalli, and D. S. Banerjee, "Fast Leiden algorithm for community detection in shared memory setting," in *Proc. 53rd Int. Conf. Parallel Process.*, 2024, pp. 11–20.

[14] W. H. Chong and L.-N. Teow, "An incremental batch technique for community detection," in *Proc. 16th Int. Conf. Inf. Fusion*, 2013, pp. 750–757.

[15] M. Halappanavar, H. Lu, A. Kalyanaraman, and A. Tumeo, "Scalable static and dynamic community detection using Grappolo," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2017, pp. 1–6.

[16] D. Zhuang, J. M. Chang, and M. Li, "DynaMo: Dynamic community detection by incrementally maximizing modularity," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 5, pp. 1934–1945, 2021.

[17] N. Zarayeneh and A. Kalyanaraman, "Delta-Screening: A fast and efficient technique to update communities in dynamic graphs," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1614–1629, 2021.

[18] S. Sahu, "A starting point for dynamic community detection with Leiden algorithm," arXiv:2405.11658, 2024.

[19] P. Chunaev, "Community detection in node-attributed social networks: A survey," *Comput. Sci. Rev.*, vol. 37, Art. no. 100286, 2020.

[20] Y. Zhang, Y. Xiong, Y. Ye, T. Liu, W. Wang, Y. Zhu, and P. S. Yu, "SEAL: Learning heuristics for community detection with generative adversarial networks," in *Proc. 26th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2020, pp. 1103–1113.

[21] V. V. Devesh Kumar, S. Sreesankar, K. Dinesan, P. B. Nair, and L. R. Deepthi, "Analyzing GNN models for community detection using graph embeddings: A comparative study," in *Proc. Int. Conf. Comput., Commun. Netw. Technol.*, 2024, pp. 1–8.

[22] Y. Liu, J. Xia, B. Wu, et al., "A survey of deep graph clustering: Taxonomy, challenge, application, and open resource," *IEEE Trans. Knowl. Data Eng.*, vol. 38, no. 6, pp. 3365–3384, 2026.

[23] S. Ji, X. Lu, M. Liu, et al., "Community-based dynamic graph learning for popularity prediction," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2023, pp. 930–940.

[24] A. Tsitsulin, J. Palowitch, B. Perozzi, and E. Müller, "Graph clustering with graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 127, pp. 1–21, 2023.

[25] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, 2009.

[26] S. Taibi, L. Toumi, and S. Bouamama, "Complex network community discovery using fast local move iterated greedy algorithm," *J. Supercomput.*, vol. 81, no. 1, Art. no. 182, 2025.

[27] Y. Liu, J. Li, Y. Chen, et al., "Revisiting modularity maximization for graph clustering: A contrastive learning perspective," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2024, pp. 1968–1979.

[28] X. Lin, R. Yang, H. Zheng, and X. Ke, "Spectral subspace clustering for attributed graphs," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 789–799.

[29] D. Kong, A. Zhang, and Y. Li, "Learning persistent community structures in dynamic networks via topological data analysis," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 8, 2024, pp. 8617–8626.

[30] B. H. Good, Y.-A. de Montjoye, and A. Clauset, "Performance of modularity maximization in practical contexts," *Phys. Rev. E*, vol. 81, Art. no. 046106, 2010.

[31] S. Fortunato and D. Hric, "Community detection in networks: A user guide," *Phys. Rep.*, vol. 659, pp. 1–44, 2016.

[32] L. Danon, A. Díaz-Guilera, J. Duch, and A. Arenas, "Comparing community structure identification," *J. Stat. Mech.: Theory Exp.*, vol. 2005, no. 9, pp. 219–228, 2005.

[33] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social Netw.*, vol. 5, no. 2, pp. 109–137, 1983.

[34] K. S. Xu and A. O. Hero, "Dynamic stochastic blockmodels for time-evolving social networks," *IEEE J. Sel. Topics Signal Process.*, vol. 8, no. 4, pp. 552–562, 2014.

[35] M. Qin, C. Zhang, Y. Gao, et al., "Towards faster graph partitioning via pre-training and inductive inference," in *Proc. IEEE High Perform. Extreme Comput. Conf.*, 2024, pp. 1–7.

[36] M. Rosvall and C. T. Bergstrom, "An information-theoretic framework for resolving community structure in complex networks," *Proc. Natl. Acad. Sci. USA*, vol. 104, no. 18, pp. 7327–7331, 2007.

[37] V. A. Traag and L. Šubelj, "Large network community detection by fast label propagation," *Sci. Rep.*, vol. 13, Art. no. 2701, 2023.

[38] J. Yang and J. Leskovec, "Defining and evaluating network communities based on ground-truth," *Knowl. Inf. Syst.*, vol. 42, no. 1, pp. 181–213, 2015.

[39] Z. Yang, R. Algesheimer, and C. J. Tessone, "A comparative analysis of community detection algorithms on artificial networks," *Sci. Rep.*, vol. 6, no. 1, Art. no. 30750, 2016.

[40] H. C. Rustamaji, W. A. Kusuma, S. Nurdiati, and I. Batubara, "Community detection with greedy modularity disassembly strategy," *Sci. Rep.*, vol. 14, Art. no. 4694, 2024.

[41] L. Cai, J. Zhou, and D. Wang, "Improving temporal smoothness and snapshot quality in dynamic network community discovery using NOME algorithm," *PeerJ Comput. Sci.*, vol. 9, Art. no. e1477, 2023.

[42] N. S. Sattar, A. Buluç, K. Z. Ibrahim, and S. Arifuzzaman, "Exploring temporal community evolution: Algorithmic approaches and parallel optimization for dynamic community detection," *Appl. Netw. Sci.*, vol. 8, Art. no. 64, 2023.

[43] J. Shang, L. Liu, F. Xie, Z. Chen, J. Miao, X. Fang, and C. Wu, "A real-time detecting algorithm for tracking community structure of dynamic networks," arXiv:1407.2683, 2014.

[44] Y. Ma, Y. Zhao, J. Wang, M. Liu, W. Shen, and Y. Ma, "Modularity-based incremental label propagation algorithm for community detection," *Appl. Sci.*, vol. 10, no. 12, Art. no. 4060, 2020.

[45] M. Seifikar, S. Farzi, and M. Barati, "C-Blondel: An efficient Louvain-based dynamic community detection algorithm," *IEEE Trans. Comput. Social Syst.*, vol. 7, no. 2, pp. 308–318, 2020.

[46] J. Xie, M. Chen, and B. K. Szymanski, "LabelRankT: Incremental community detection in dynamic networks via label propagation," in *Proc. Workshop Dyn. Netw. Manage. Mining*, 2013, pp. 25–32.

[47] J. Han, W. Li, L. Zhao, Z. Su, Y. Zou, and W. Deng, "Community detection in dynamic networks via adaptive label propagation," *PLOS ONE*, vol. 12, no. 11, Art. no. e0188655, 2017.

[48] A. Bovet, J.-C. Delvenne, and R. Lambiotte, "Flow stability for dynamic community detection," *Sci. Adv.*, vol. 8, no. 19, Art. no. eabj3063, 2022.

[49] Z. Sun, Y. Sun, X. Chang, F. Wang, Z. Pan, G. Wang, and J. Liu, "Dynamic community detection based on the Matthew effect," *Physica A*, vol. 597, Art. no. 127315, 2022.

[50] M. Mazza, G. Cola, and M. Tesconi, "Modularity-based approach for tracking communities in dynamic social networks," *Knowl.-Based Syst.*, vol. 281, Art. no. 111067, 2023.

[51] V. Brabant, A. Bonifati, and R. Cazabet, "Discovering communities in continuous-time temporal networks by optimizing L-modularity," in *Proc. IEEE Int. Conf. Data Mining*, 2025, pp. 1065–1074.

[52] N. Djurdjevac Conrad, E. Tonello, J. Zonker, and H. Siebert, "Detection of dynamic communities in temporal networks with sparse data," *Appl. Netw. Sci.*, vol. 10, Art. no. 1, 2025.

[53] F. Blašković, T. O. F. Conrad, S. Klus, and N. Djurdjevac Conrad, "Random walk based snapshot clustering for detecting community dynamics in temporal networks," *Sci. Rep.*, vol. 15, Art. no. 24414, 2025.

[54] K. Hanauer, M. Henzinger, and C. Schulz, "Recent advances in fully dynamic graph algorithms: A quick reference guide," *ACM J. Exp. Algorithmics*, vol. 27, Art. no. 1.11, 2022.

[55] S. Sahu, "DF Louvain: Fast incrementally expanding approach for community detection on dynamic graphs," arXiv:2404.19634, 2024.

[56] S. Sahu, K. Kothapalli, and D. S. Banerjee, "Parallel multicore algorithms for community detection in dynamic graphs," *Int. J. Netw. Comput.*, vol. 15, no. 1, pp. 2–31, 2025.

[57] N. S. Sattar, K. Z. Ibrahim, A. Buluç, and S. Arifuzzaman, "DyG-DPCD: A distributed parallel community detection algorithm for large-scale dynamic graphs," *Int. J. Parallel Program.*, vol. 53, no. 1, Art. no. 4, 2025.

[58] D. J. DiTursi, G. Ghosh, and P. Bogdanov, "Local community detection in dynamic networks," in *Proc. IEEE Int. Conf. Data Mining*, 2017, pp. 847–852.

[59] D. Christopoulos, G. Baltsou, and K. Tsichlas, "Local community detection in graph streams with anchors," *Information*, vol. 14, no. 6, Art. no. 332, 2023.

[60] J. Liu, Y. Shao, and S. Su, "Multiple local community detection via high-quality seed identification over both static and dynamic networks," *Data Sci. Eng.*, vol. 6, pp. 249–264, 2021.

[61] S. Boudebza, R. Cazabet, F. Azouaou, and O. Nouali, "OLCPM: An online framework for detecting overlapping communities in dynamic social networks," *Comput. Commun.*, vol. 123, pp. 36–51, 2018.

[62] C. Liu, Y. Yang, Y. Ding, et al., "DAG: Deep adaptive and generative K-free community detection on attributed graphs," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2024, pp. 5454–5465.

[63] T. Zhang, Y. Xiong, J. Zhang, et al., "CommDGI: Community detection oriented deep graph infomax," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 1843–1852.

[64] A. Bhowmick, M. Kosan, Z. Huang, A. Singh, and S. Medya, "DGCLUSTER: A neural framework for attributed graph clustering via modularity maximization," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 10, 2024, pp. 11069–11077.

[65] J. Zhang, H. Peng, L. Sun, et al., "Unsupervised graph clustering with deep structural entropy," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 3752–3763.

[66] M. Qin, C. Zhang, Y. Gao, W. Zhang, and D.-Y. Yeung, "Pre-train and refine: Towards higher efficiency in K-agnostic community detection without quality degradation," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2024, pp. 2467–2478.

[67] C. Gkartzios, E. Pitoura, and P. Tsaparas, "Modularity-fair deep community detection," in *Proc. IEEE Int. Conf. Data Mining*, 2025, pp. 287–296.